

Deep Learning Assignment 2

1 Introduction

This assignment consists of two parts:

- **Part 1:** Image segmentation using a U-Net model
- **Part 2:** A sequence modeling task using a Recurrent Neural Network (RNN)

In this assignment, you will implement deep learning models for both spatial and sequential data. Through this assignment, you will gain hands-on experience with both spatial (image) and sequential data modeling. If you have any questions, contact **hslee0608@postech.ac.kr**.

2 Image Segmentation

2.1 What is Image Segmentation

Image segmentation is the task of dividing an image into meaningful regions. In binary segmentation, each pixel is classified into one of two classes: foreground (object of interest) and background. The output of a segmentation model is a pixel-wise prediction map with the same spatial resolution as the input image.

2.2 Dataset

For the image segmentation task, we use the **Oxford-IIIT Pet dataset**. This dataset contains pet images and their corresponding pixel-wise segmentation masks. Each sample consists of:

- an input image
- a segmentation mask with the same spatial resolution

After preprocessing and loading with the DataLoader, each batch is represented as:

```
images, masks = next(iter(train_loader))
```

where:

- **images:** input images of shape (B, C, H, W)
 - B : batch size
 - C : number of channels (3 for RGB images)
 - H, W : height and width
- **masks:** segmentation masks of shape $(B, 1, H, W)$
 - each pixel is a class label
 - 0: background, 1: foreground

2.3 U-Net

U-Net is a convolutional neural network designed for image segmentation. It follows an **encoder-decoder** structure, where the model first compresses the input image into a set of high-level features and then reconstructs a segmentation map.

Encoder (Contracting Path) The encoder gradually reduces the spatial resolution of the input while increasing the number of feature channels. This is typically done using convolution layers followed by pooling operations. The purpose of the encoder is to capture semantic information, such as the presence of objects and their general structure.

Decoder (Expanding Path) The decoder gradually restores the spatial resolution using up-sampling operations (e.g., transposed convolutions). It transforms the compressed features back into a full-resolution segmentation map. The decoder focuses on recovering spatial details, such as object boundaries.

Skip Connections A keyfeature of U-Net is the use of skip connections. At each level, feature maps from the encoder are concatenated with the corresponding feature maps in the decoder. These connections help:

- preserve spatial information
- improve localization accuracy
- recover fine details lost during downsampling

The overall architecture of U-Net is illustrated at Fig.1. The model follows an encoder-decoder structure with skip connections that transfer feature maps from the encoder to the decoder.

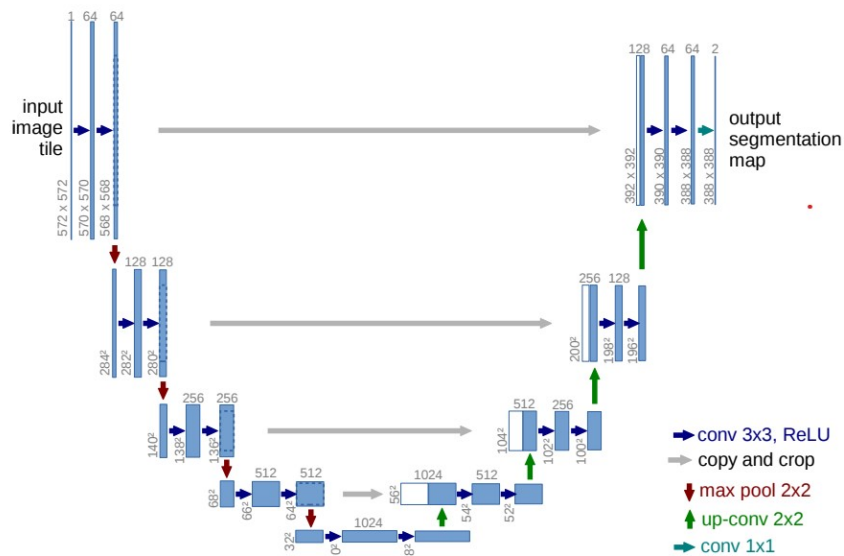


Figure 1: U-Net architecture

2.4 Implementation Tasks

- **Model Implementation:** You are given a skeleton code for the U-Net architecture. Your task is to complete the implementation.
- **Training:** After implementing the model, you will train it to perform binary segmentation on the given dataset.
- **Evaluation:** Evaluate on the test set using:
 - **Pixel Accuracy:** Pixel accuracy measures the proportion of correctly classified pixels over all pixels. It is defined as:

$$\text{Pixel Accuracy} = \frac{\text{Number of correct pixels}}{\text{Total number of pixels}}$$

- **Dice Coefficient:** The Dice coefficient measures the overlap between the predicted mask and the ground-truth mask. When P denotes the set of predicted foreground pixels and G denotes the set of ground-truth foreground pixels, the Dice score is defined as:

$$\text{Dice} = \frac{2|P \cap G|}{|P| + |G|}$$

- **Mean Intersection over Union (mIoU):** IoU measures the ratio between the intersection and the union of prediction and ground truth:

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|}$$

For multi-class segmentation with C classes, IoU is computed for each class, and mIoU is defined as the average:

$$\text{mIoU} = \frac{1}{C} \sum_{c=1}^C \text{IoU}_c$$

For binary segmentation, the mIoU is computed by averaging the IoU scores of the foreground and background classes.

- **Visualization:** Visualize the prediction with the ground-truth mask.

3 Sequence Modeling with RNN

3.1 What is Sequence Modeling

Sequence modeling is the task of processing ordered data, where the prediction depends not only on the current input but also on previous elements in the sequence.

3.2 Dataset

For the sequence modeling task, we use a **character-level name classification dataset** from the PyTorch tutorial. Each sample consists of a person's name and its corresponding category (e.g., nationality). Each sample contains:

- an input name represented as a sequence of characters

- a label indicating its category

Before being fed into the model, each name is converted into a sequence of character indices. After preprocessing and loading with the DataLoader, each batch is represented as:

```
names, labels, lengths = next(iter(train_loader))
```

where:

- **names:** padded sequences of shape (B, T)
 - B : batch size
 - T : maximum sequence length in the batch
- **labels:** class indices of shape $(B,)$
- **lengths:** original sequence lengths of shape $(B,)$

3.3 Recurrent Neural Networks (RNNs)

Unlike standard feedforward networks, recurrent neural networks (RNNs) are designed to handle sequential data by maintaining a hidden state that is updated at each time step. This allows the model to accumulate information over time.

An RNN processes a sequence one element at a time. At each time step t , the RNN updates its hidden state h_t :

$$h_t = \tanh(x_t W_{xh} + h_{t-1} W_{hh} + b_h)$$

After processing the entire sequence, the final hidden state can be used to make a prediction:

$$y = h_T W_{hy} + b_y$$

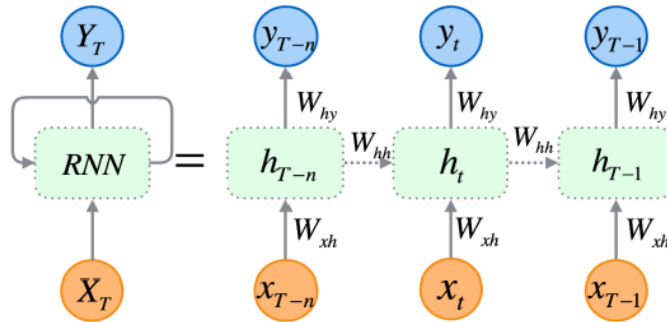


Figure 2: RNN architecture

3.4 Implementation Tasks

- **Model Implementation:** You are given a skeleton code for the RNN model. Your task is to complete the implementation.
- **Training:** After implementing the model, you will train it to classify each input name into its corresponding category.

- **Evaluation:** Evaluate on the test set using:

- **Accuracy:** Accuracy measures the proportion of correctly classified samples:

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{\text{Total number of samples}}$$

- **Macro-F1 Score:** The F1 score is the harmonic mean of precision and recall:

$$\text{F1} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}}$$

where

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

(TP = True Positive, FP = False Positive, and FN = False Negative.)

In multi-class classification, the **Macro-F1** score is computed by first calculating the F1 score for each class independently, and then averaging them:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C F1_c$$

where C is the number of classes.

- **Prediction:** Print several example predictions along with the ground-truth labels. Analyze the results.